



PROYECTO FIN DE GRADO

Yummy!

2º DAM. CESUR

Tutor: Gonzalo Pérez Crespo

Marina Maestre Sánchez
Curso académico: 2025–2026

1. Tabla de contenido

2.	Resumen.....	3
3.	Introducción.....	3
3.1.	Contexto y motivación.....	3
3.2.	Objetivos.....	4
3.3.	Alcance y Requisitos del Proyecto.....	4
4.	Análisis y planificación.....	5
4.1.	Necesidad detectada.....	5
4.2.	Relación con proyectos tipo.....	5
4.3.	Contexto personal y enfoque del proyecto.....	5
4.4.	Requisitos no funcionales.....	5
4.5.	Usuarios y Roles.....	6
4.6.	Casos de Uso.....	6
4.7.	Diagramas UML.....	6
5.	Desarrollo e Implementación.....	8
5.1.	Entorno de desarrollo.....	8
5.2.	Estructura del proyecto.....	8
5.3.	Gestión del Repositorio en GitHub.....	8
5.4.	Configuración del Backend y Dependencias.....	9
5.5.	Variables de entorno.....	9
5.6.	Servidor y arquitectura básica.....	9
5.7.	Sistema de autenticación.....	10
5.8.	Conexión con la base de datos.....	11
5.9.	Validación con Postman.....	11
5.10.	Modelo de datos.....	12
5.11.	Lógica de gestión de recetas.....	12
5.12.	Rutas y protección de acceso.....	13
5.13.	Validación final del sistema de autenticación y seguridad.....	13
5.14.	Estado del backend.....	14
5.15.	Estrategia de despliegue y persistencia en la nube.....	15
6.	Desarrollo de la Interfaz de Usuario.....	15
6.1.	Elección de React y Configuración del Entorno.....	15
6.2.	Arquitectura de Componentes.....	15
6.3.	Sistema de rutas.....	16
6.4.	Conexión entre el frontend y el backend.....	16

6.5.	Diseño Visual y Experiencia de Usuario (UX).	17
6.6.	Validación de Accesibilidad.	19
7.	Manual de Usuario.	20
7.1.	Acceso y Registro	20
7.2.	Gestión de Recetas (Dashboard)	20
7.3.	Creación y Edición de platos.	21
7.4.	Cocinando con el Modo Lectura.	22
7.5.	Sección Explorar: Recetas de la Comunidad.	22
8.	Valoración Económica y Temporal.	23
8.1.	Estimación temporal.	23
8.2.	Coste estimado.	24
9.	Conclusiones y Futuras Líneas de Trabajo.	24
9.1.	Valoración del proyecto.	24
9.2.	Mejoras futuras.	25
10.	Bibliografía y Webgrafía.	25

2. Resumen.

“Yummy! – Gestor de Recetas” es un proyecto intermodular de 2º DAM centrado en el desarrollo de una aplicación web full-stack para la gestión de recetas personales. Su objetivo principal es permitir al usuario crear, consultar, modificar y eliminar recetas de forma sencilla, manteniendo un recetario digital accesible desde cualquier dispositivo.

El proyecto combina dos dimensiones. Por un lado, la parte técnica, basada en la construcción de un sistema CRUD utilizando Node.js, Express y una base de datos NoSQL. Por otro, una parte más personal, orientada a conservar recetas familiares con influencia andaluza y marroquí (Node.js Foundation, s.f.).

La accesibilidad ha sido un aspecto clave durante todo el desarrollo, buscando una interfaz clara, legible y fácil de usar para cualquier tipo de usuario, independientemente de su experiencia con la tecnología.

3. Introducción.

3.1. Contexto y motivación.

Yummy! nace de algo muy cotidiano: la dificultad de mantener las recetas organizadas cuando están repartidas entre libretas, notas del móvil, fotos o incluso recuerdos. Es una situación muy común, pero en mi caso tiene además un componente personal importante relacionado con mi entorno familiar.

Este proyecto surge con la idea de conservar ese recetario, dándole forma digital en un espacio organizado, accesible y fácil de usar. Al mismo tiempo, representa el cierre de mi formación en el ciclo de Desarrollo de Aplicaciones Multiplataforma, integrando conocimientos de programación, bases de datos y desarrollo web.

Desde el inicio, la accesibilidad no se ha planteado como un añadido, sino como una parte fundamental del proyecto. La intención es que cualquier persona pueda utilizar la aplicación con facilidad, independientemente de su edad o experiencia digital.

Para ello, he trabajado con contrastes adecuados, tipografías legibles y una estructura clara, priorizando siempre la comodidad de uso. Este enfoque permite combinar una buena experiencia de usuario con un desarrollo técnico sólido.

3.2. Objetivos.

Objetivo general:

- Desarrollar una aplicación web full-stack para la gestión de recetas personales mediante un sistema CRUD completo y funcional.

Objetivos específicos:

- Diseñar una interfaz accesible, clara y usable.
- Implementar una API REST con Node.js y Express.
- Definir y modelar una base de datos NoSQL.
- Aplicar buenas prácticas de diseño y accesibilidad.
- Controlar el desarrollo mediante Git y GitHub
- Documentar el proceso en todas sus fases.

3.3. Alcance y Requisitos del Proyecto.

El proyecto se centra en el desarrollo de una aplicación funcional que permita gestionar recetas personales. Las funcionalidades principales incluyen la creación, consulta, edición y eliminación de recetas, así como su organización mediante filtros y categorías.

Además, se han tenido en cuenta aspectos como la usabilidad, la accesibilidad y la posibilidad de ampliar el sistema en futuras versiones. La siguiente tabla resume las capacidades operativas de la aplicación:

ID	Requisito funcional	Descripción y Alcance
RF-01	Gestión de Usuarios	Registro, login y cifrado de contraseñas mediante JWT
RF-02	CRUD de Recetas	Crear, leer, actualizar y eliminar recetas personales
RF-03	Buscador y Filtros	Filtrado por título, ingredientes y categorías en tiempo real
RF-04	Modo Lectura	Interfaz simplificada y accesible para el proceso de cocina
RF-05	Privacidad	Opción de marcar recetas como privadas o públicas (Explorar)
RF-06	Feedback Visual	Notificaciones de carga, errores y animación de éxito

4. Análisis y planificación.

4.1. Necesidad detectada.

Hoy en día, muchas personas guardan sus recetas de forma dispersa: en libretas, notas del móvil, fotos o incluso en mensajes. Esto hace más difícil encontrarlas cuando se necesitan, mantenerlas organizadas o evitar que se pierdan con el tiempo.

Yummy! surge como una solución a este problema, ofreciendo un espacio único donde centralizar y gestionar un recetario personal de forma sencilla y accesible.

4.2. Relación con proyectos tipo.

El sistema se corresponde con una aplicación CRUD de contenido, uno de los modelos más habituales en el desarrollo web y multiplataforma. Este tipo de proyecto permite trabajar: diseño de modelo de datos, lógica backend, exposición de endpoints API, interfaz responsive, accesibilidad y control de versiones.

4.3. Contexto personal y enfoque del proyecto.

Además de la parte técnica, este proyecto tiene un componente personal importante. La idea de crear un recetario digital nace de la intención de conservar recetas familiares y facilitar su uso en el día a día.

Por este motivo, se ha puesto especial atención en que la aplicación sea clara y fácil de utilizar, pensando en usuarios con distintos niveles de experiencia digital, incluyendo personas que no están acostumbradas a usar este tipo de herramientas.

4.4. Requisitos no funcionales.

Los requisitos no funcionales definen cómo debe comportarse el sistema y qué calidad debe tener.

- **Accesibilidad.** La aplicación sigue criterios básicos de accesibilidad, como buen contraste de colores, textos legibles y una estructura clara.
- **Usabilidad.** Se busca una navegación sencilla, con elementos visibles y fáciles de entender.
- **Rendimiento.** Los tiempos de carga deben ser razonables para no afectar a la experiencia del usuario.
- **Portabilidad.** La aplicación debe funcionar correctamente en navegadores actuales.

- Escalabilidad. El sistema está preparado para poder añadir nuevas funcionalidades en el futuro.
- Mantenibilidad. El código seguirá una estructura clara, con carpetas organizadas por componentes y control de versiones mediante GitHub.
- Diseño responsive. La interfaz se adapta a diferentes tamaños de pantalla.
- Consistencia visual. Se mantiene una estética uniforme, clara y legible en toda la aplicación.

4.5. Usuarios y Roles.

En la versión actual del proyecto existe un único tipo de usuario, denominado “usuario general”. Este usuario es la persona que utiliza la aplicación para gestionar sus recetas personales. Puede tener distintos niveles de experiencia digital, por lo que la aplicación debe ser fácil de usar en todos los casos. Los objetivos principales del usuario son:

- Guardar recetas sin perderlas.
- Tenerlas organizadas.
- Acceder a ellas de forma rápida y sencilla.

4.6. Casos de Uso.

Los casos de uso describen las interacciones principales que el usuario puede realizar dentro de la aplicación:

- Crear receta. El usuario introduce los detalles de la receta y la guarda en la base de datos.
- Consultar receta. El usuario visualiza el listado general y selecciona una receta para ver sus detalles.
- Editar receta. El usuario modifica la información existente y guarda los cambios.
- Eliminar receta. El usuario borra una receta, previa confirmación.
- Buscar recetas. El usuario introduce un término y el sistema filtra las recetas coincidentes.
- Filtrar por categoría. El usuario selecciona una categoría para visualizar solo las recetas relacionadas con esta.
- Explorar recetas. El usuario podrá acceder a un listado de recetas públicas que otros chefs han querido compartir.

4.7. Diagramas UML.

Para representar de manera visual la estructura y funcionamiento he usado diferentes diagramas UML (realizados con la herramienta DIA).

- Diagrama de Casos de Uso. Muestra las acciones principales que el usuario puede realizar en la aplicación.

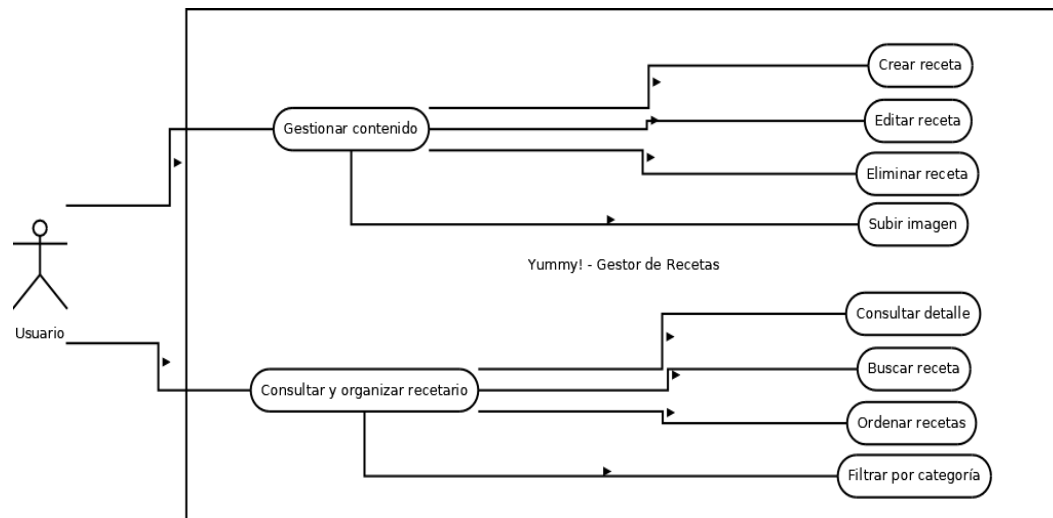


Ilustración 1. Diagrama UML de Casos de Uso.

- Diagrama de clases. Representa los elementos clave de la lógica interna de nuestra aplicación. Este diagrama servirá como base para el diseño backend.

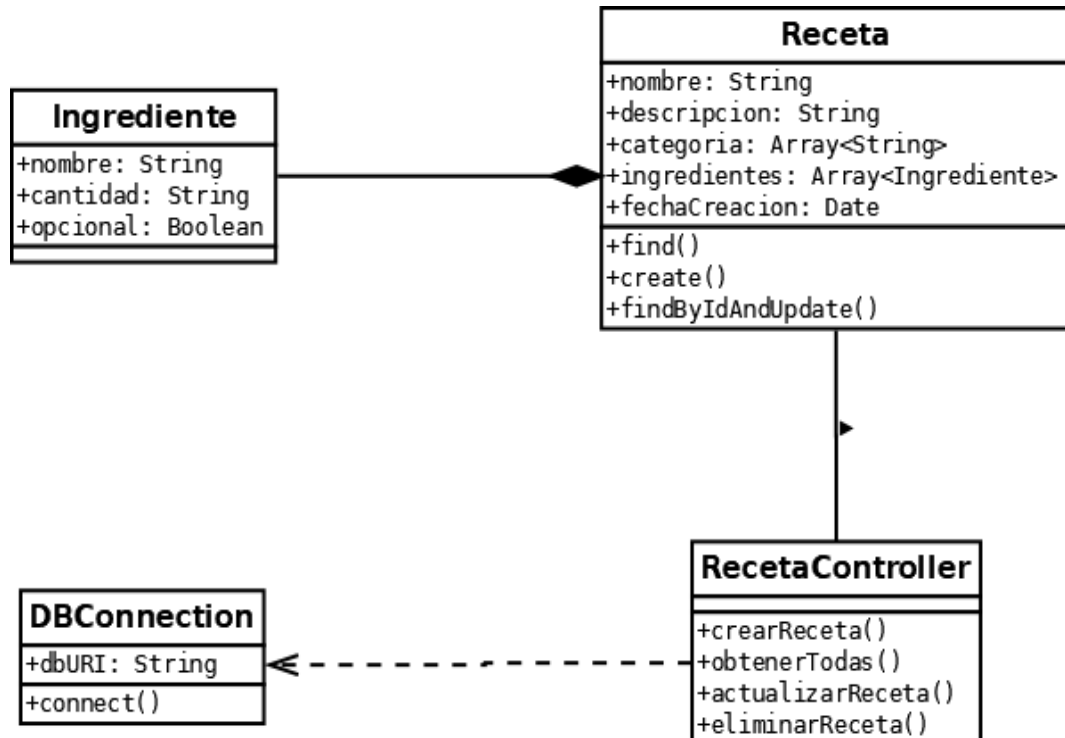


Ilustración 2. Diagrama UML de clases.

- Diagrama de secuencia. describe el flujo de interacción entre usuario, API y base de datos.

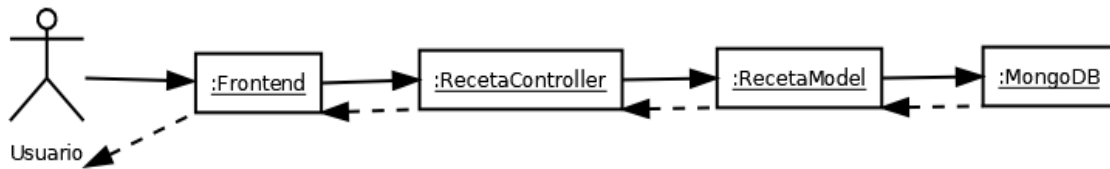


Ilustración 3. Diagrama UML de secuencia

5. Desarrollo e Implementación.

5.1. Entorno de desarrollo.

Antes de comenzar con la implementación, he preparado un entorno de trabajo sencillo y organizado. He utilizado Visual Studio Code como entorno de desarrollo y Node.js como base para el backend. Para el control de versiones, he trabajado con Git y GitHub, lo que me ha permitido llevar un seguimiento claro de los cambios realizados durante el proyecto (Microsoft, s.f.).

```

Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\marin>node -v
v22.16.0

C:\Users\marin>npm -v
10.9.2

C:\Users\marin>
  
```

Ilustración 4. Verificación de versiones instaladas de Node.js.

5.2. Estructura del proyecto.

El proyecto se ha organizado en tres carpetas principales:

- Backend: contiene toda la lógica del servidor.
- Frontend: incluye la interfaz desarrollada con React.
- Docs: documentación del proyecto.

Esta estructura permite separar claramente cada parte y facilita el mantenimiento del código.

5.3. Gestión del Repositorio en GitHub.

Para garantizar el control del desarrollo, creé el repositorio remoto “Yummy-Gestor-Recetas” en GitHub. Durante todo el desarrollo he ido subiendo al repositorio las diferentes versiones.

El código fuente del proyecto se encuentra disponible en el repositorio público de GitHub (GitHub, s.f.):

<https://github.com/MarinaMaestre/Yummy-Gestor-Recetas>.

5.4. Configuración del Backend y Dependencias.

Para el desarrollo del Backend he seleccionado librerías específicas que garantizan seguridad y eficiencia en la gestión de datos:

Dependencia	Función Principal en el Proyecto
Express.js	Framework para la creación de la API REST y gestión de rutas
Mongoose	Modelado de objetos de MongoDB para gestionar esquemas de datos
Bcryptjs	Cifrado de contraseñas mediante hashing antes del guardado
JSON Web Token	Generación de tokens de sesión para autenticación segura
Dotenv	Gestión de variables de entorno y protección de credenciales
Cors	Configuración de permisos para la comunicación con el Frontend

5.5. Variables de entorno.

Para evitar exponer información sensible, he utilizado un archivo .env donde se almacenan datos como el puerto del servidor, la conexión a la base de datos y la clave secreta de los tokens.

Esto permite mantener el código más seguro y facilita su reutilización en otros entornos.

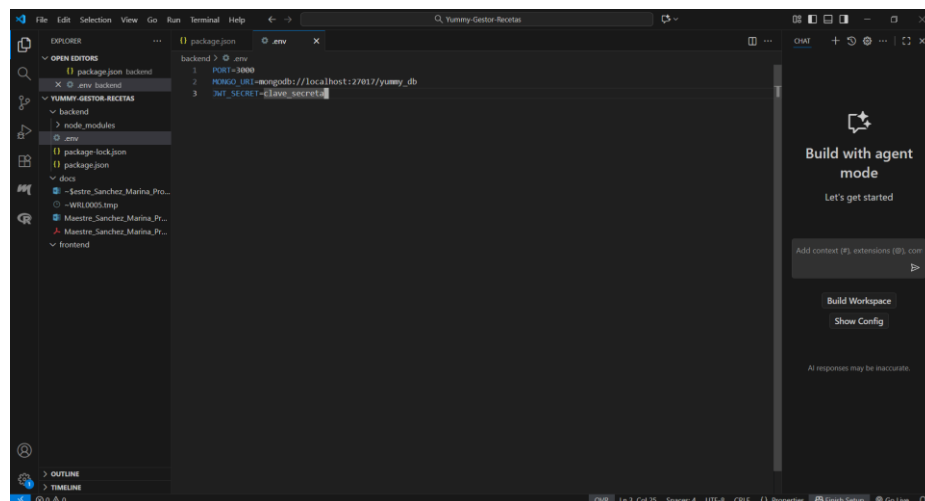


Ilustración 5. Configuración de variables de entorno en el archivo .env.

5.6. Servidor y arquitectura básica.

El archivo principal del backend es server.js, que se encarga de iniciar el servidor y configurar Express.

En este punto, he comprobado que el servidor arranca correctamente y queda a la espera de peticiones, lo que marca el inicio del funcionamiento real de la aplicación.

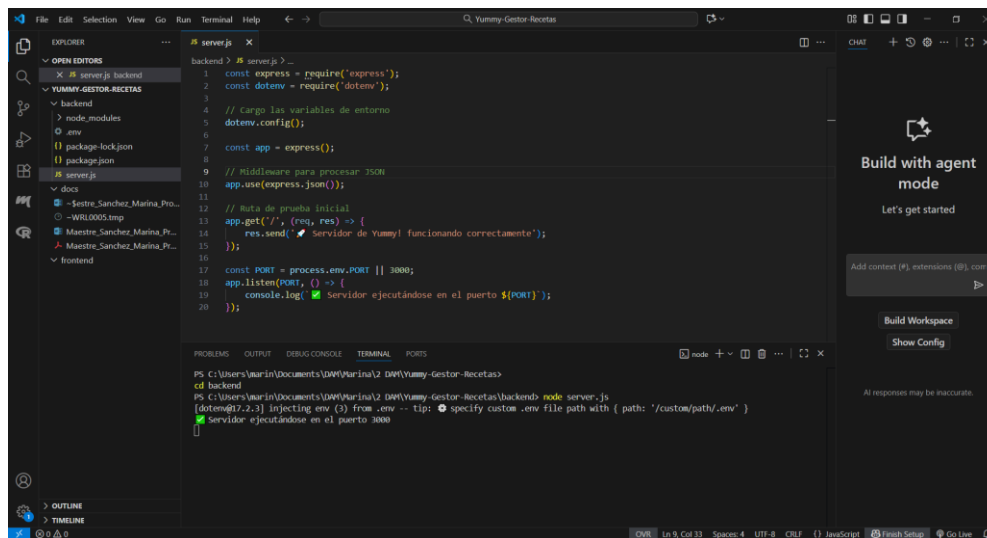


Ilustración 6. Ejecución exitosa del servidor Express y validación de variables de entorno.

5.7. Sistema de autenticación.

Para proteger el acceso a la aplicación, he implementado un sistema de autenticación basado en JSON Web Tokens (JWT).

Las contraseñas se almacenan cifradas mediante bcryptjs, evitando guardar información sensible en texto plano (Bcryptjs, s.f.).

Además, he desarrollado un middleware que valida el token en cada petición protegida, garantizando que solo los usuarios autenticados puedan acceder a sus recetas.

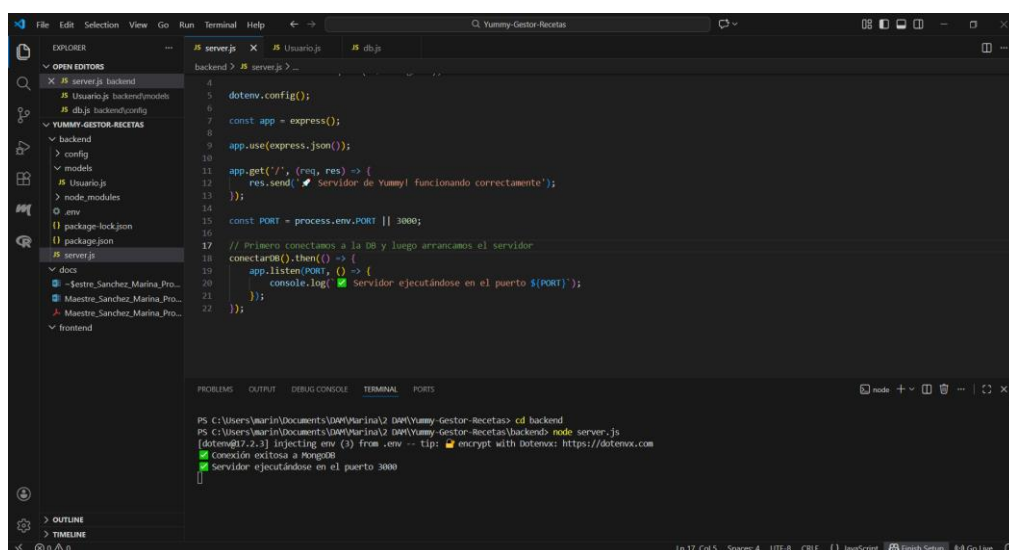


Ilustración 7. Validación final de conexión exitosa entre el servidor y MongoDB

5.8. Conexión con la base de datos.

Para el almacenamiento de datos, he utilizado MongoDB junto con Mongoose. He creado un módulo específico para gestionar la conexión, lo que permite mantener el código organizado. Tras varias pruebas, he comprobado que la conexión funciona correctamente y que el sistema puede guardar y recuperar información sin problemas (MongoDB, s.f.); (Mongoose, s.f.).

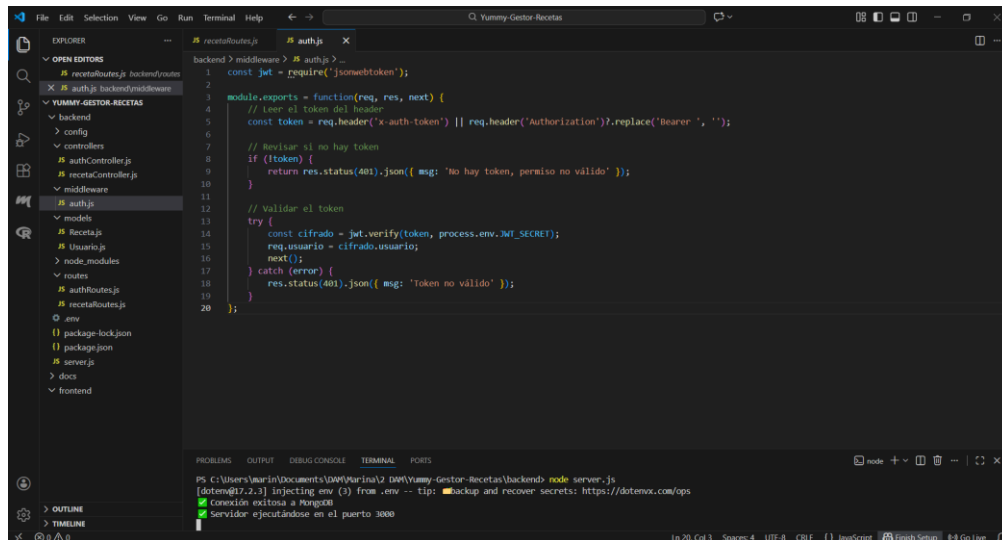


Ilustración 8. Código middleware de autenticación.

5.9. Validación con Postman.

Antes de desarrollar el frontend, he comprobado el funcionamiento del backend utilizando Postman. Esto me ha permitido enviar peticiones directamente al servidor y validar que las rutas responden correctamente (Postman, s.f.).

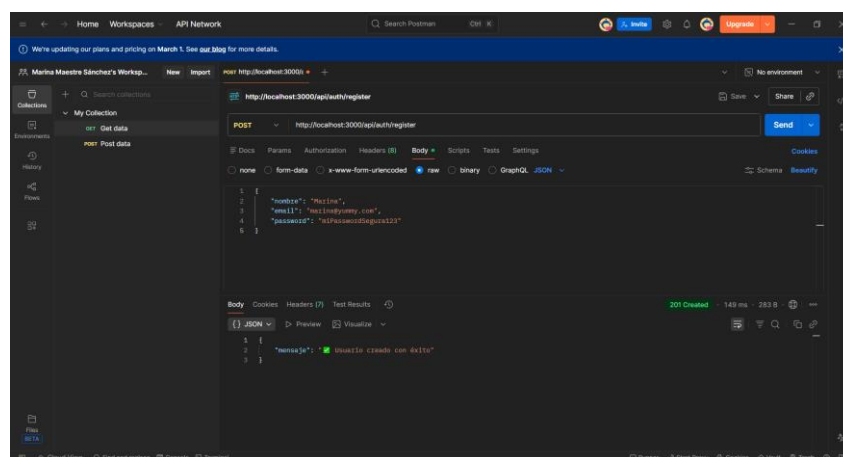


Ilustración 9. Creación de primer usuario desde Postman

5.10. Modelo de datos.

He definido un modelo de datos para las recetas que incluye:

- Título.
- Descripción.
- Ingredientes.
- Pasos.
- Usuario asociado.
- Pública o privada.

Esto permite estructurar la información de forma clara y facilita su gestión dentro de la aplicación.

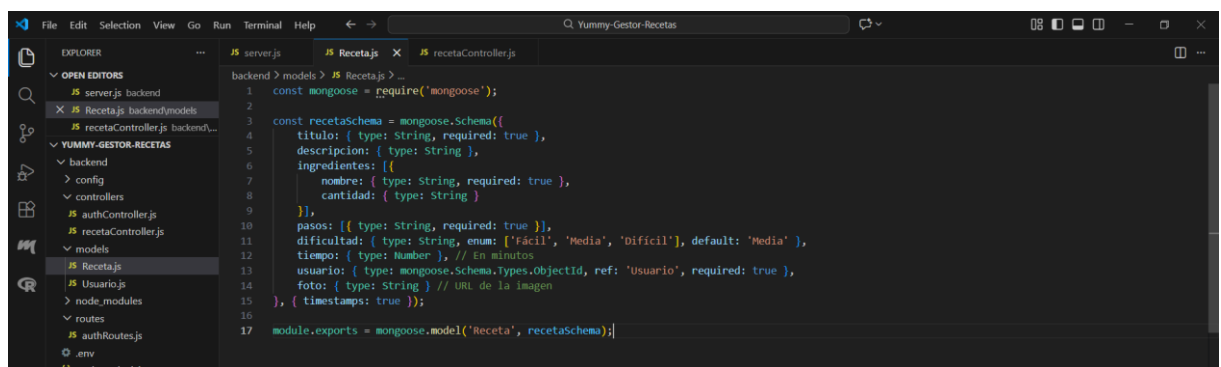


Ilustración 10. Código del modelo de datos para recetas.

5.11. Lógica de gestión de recetas.

He desarrollado un controlador encargado de gestionar las operaciones principales del sistema:

- Crear recetas.
- Consultar recetas.
- Editar recetas.
- Eliminar recetas.

Además, el sistema asocia automáticamente cada receta con el usuario que la crea, lo que permite mantener un recetario organizado por usuario.

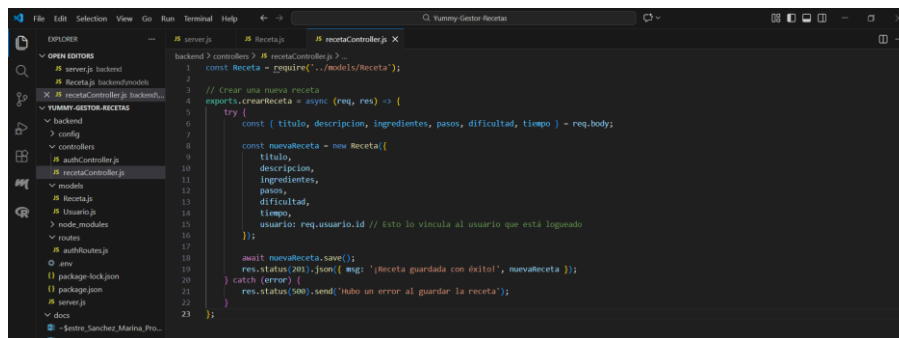


Ilustración 11. Código del controlador de recetas.

5.12. Rutas y protección de acceso.

Las rutas del backend conectan las peticiones del cliente con la lógica del sistema. Para la gestión de estas rutas y la comunicación eficiente entre las capas de la aplicación, se ha utilizado el framework (Express.js, s.f.). En aquellas rutas que requieren autenticación, he aplicado el middleware de verificación de token, evitando así accesos no autorizados y garantizando la seguridad de los datos de cada usuario.

Para mantener un código limpio y escalable, se ha utilizado el Router de Express.js, lo que permite crear manejadores de rutas modulares y montables. De esta forma, la lógica de las recetas y la de los usuarios se gestionan de manera independiente, facilitando el mantenimiento del sistema (Express.js, s.f.).

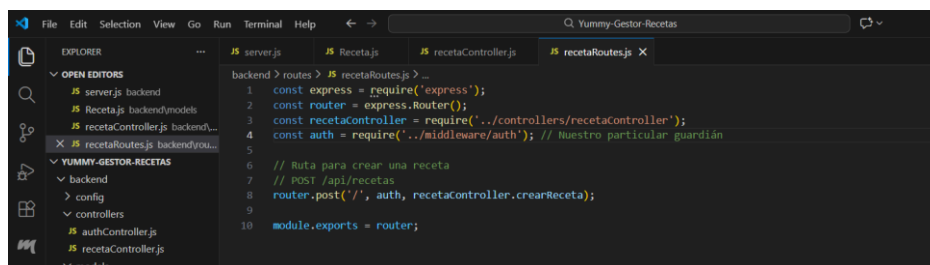


Ilustración 12. Configuración de rutas para el módulo de recetas.

5.13. Validación final del sistema de autenticación y seguridad.

Antes de conectar el frontend, he comprobado el funcionamiento de la API utilizando Postman. Esto me ha permitido validar que el servidor responde correctamente tanto en casos normales como en situaciones de error.

Escenario de prueba	Resultado esperado	Estado
Registro de nuevo usuario	Usuario creado correctamente en DB	OK
Login con datos válidos	Generación de Token JWT	OK

Login con datos incorrectos	Error 401 (No autorizado)	OK
Crear receta sin Token	Acceso denegado	OK
CRUD completo autenticado	Funcionamiento correcto de la API	OK

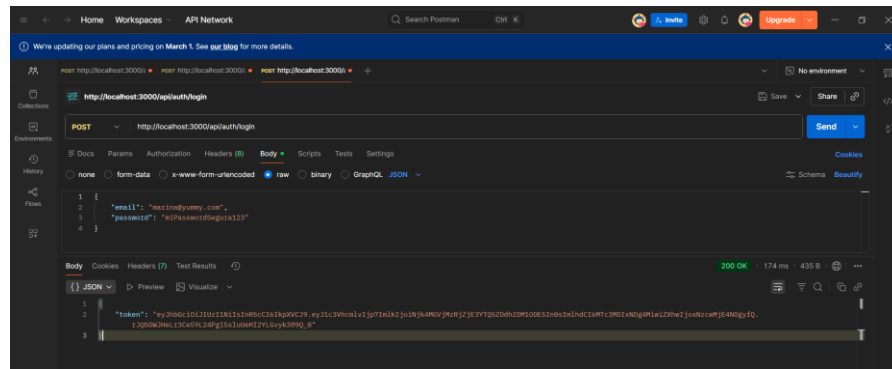


Ilustración 13. Generación exitosa de JSON Web Token.

Además de las pruebas manuales, he implementado scripts de validación automáticos (tests) dentro de Postman. Estos scripts permiten verificar de forma automática que el servidor responde con un estado OK y que el formato de los datos devueltos es el correcto, asegurando la estabilidad del sistema ante cualquier cambio en el código.

5.14. Estado del backend.

Tras estas comprobaciones, el backend queda completamente operativo, preparado para ser utilizado por la interfaz de usuario.

El sistema es capaz de recibir datos, procesarlos y almacenarlos de forma persistente en la base de datos.

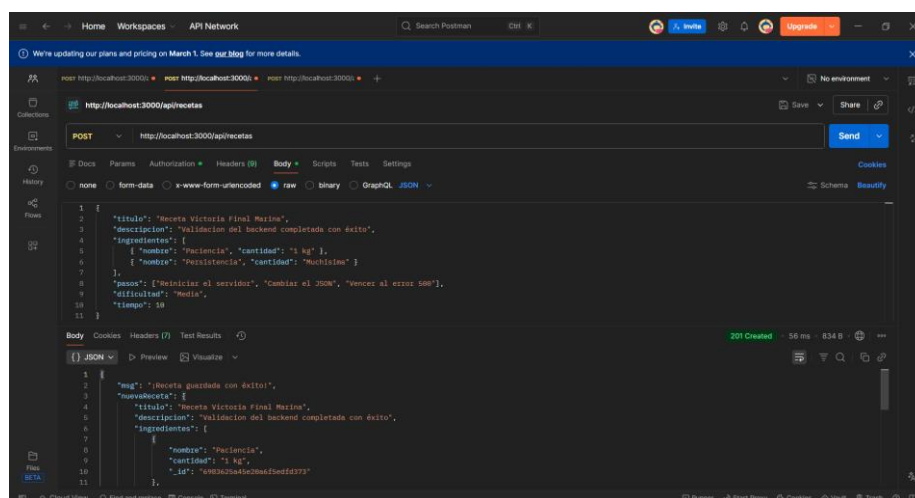


Ilustración 14. Confirmación final de guardado de receta en MongoDB.

5.15. Estrategia de despliegue y persistencia en la nube.

El proyecto se ha diseñado para operar en un entorno real. La base de datos ya funciona de forma remota mediante MongoDB Atlas, lo que asegura que las recetas no se pierdan. Para la puesta en marcha definitiva, el plan contempla desplegar el servidor en Render y la interfaz de usuario en Vercel, aprovechando la conexión con GitHub para que la aplicación sea accesible mediante HTTPS.

Esta arquitectura permite escalar la aplicación fácilmente y facilita su mantenimiento en un entorno real de producción.

6. Desarrollo de la Interfaz de Usuario.

Cabe destacar que, para evitar conflictos de red con el entorno de React, el puerto del servidor Backend se configuró finalmente en el 5000, dejando el 3000 para la ejecución del Frontend.

6.1. Elección de React y Configuración del Entorno.

Para el desarrollo del frontend he utilizado React, ya que me permite construir una interfaz dinámica y fluida sin necesidad de recargar la página constantemente. (Meta Open Source, s.f.)

He iniciado el proyecto con Create React App, lo que me ha facilitado una estructura base sobre la que trabajar de forma ordenada. Gracias al uso de npm start, he podido visualizar los cambios en tiempo real, algo que me ha resultado especialmente útil durante el ajuste de componentes y estilos.

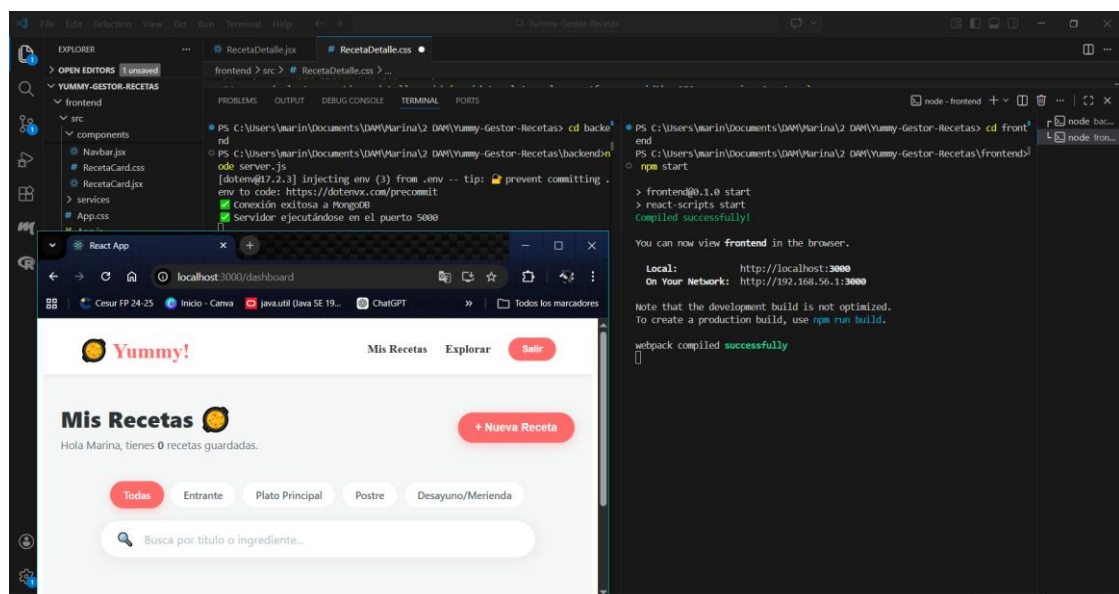


Ilustración 15. Entorno de ejecución simultáneo: Servidor de API (Node.js) y Cliente Web (React) operativos en puertos locales.

6.2. Arquitectura de Componentes.

La interfaz se ha desarrollado siguiendo una arquitectura basada en componentes reutilizables. Dentro de la carpeta src, he separado:

- components: elementos reutilizables como barra de navegación o tarjetas de receta.
- pages: vistas principales como Dashboard, Explorar o Detalle de receta.

Esta organización evita duplicidades y facilita futuras modificaciones.

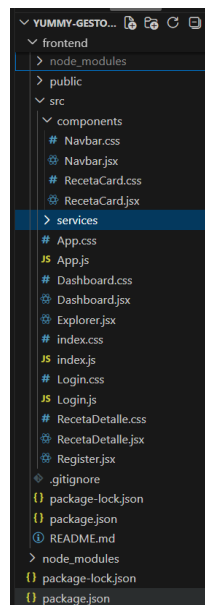


Ilustración 16. Organización del árbol de directorios en React y modularización por componentes.

6.3. Sistema de rutas.

La navegación entre pantallas se ha gestionado con react-router-dom.

Desde el archivo principal App.js, he definido las rutas públicas y privadas, redirigiendo al usuario al login cuando no dispone de un token válido.

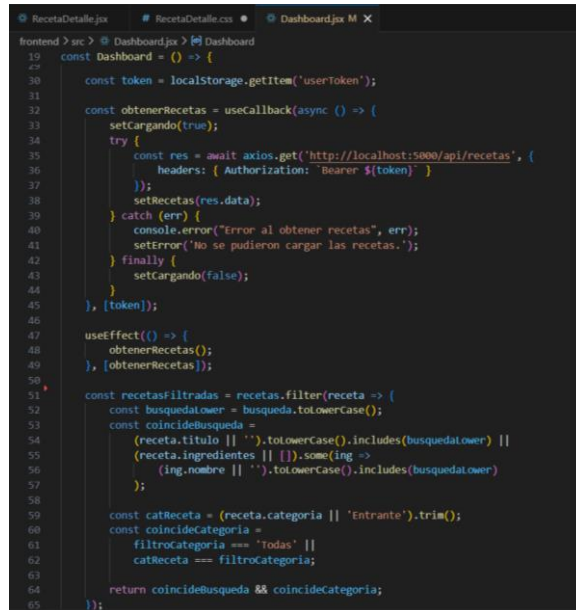
Esto permite proteger el contenido personal y mantener la privacidad de las recetas.

6.4. Conexión entre el frontend y el backend.

Para conectar la interfaz con la API he utilizado Axios, realizando peticiones HTTP al backend para consultar, crear, editar y eliminar recetas (Axios Project, s.f.).

El token JWT se recupera desde localStorage y se envía en cada petición protegida dentro de la cabecera de autorización.

Además, mediante `useEffect`, la aplicación carga automáticamente las recetas del usuario al acceder al Dashboard, mejorando la experiencia de uso.



```

30 const token = localStorage.getItem('userToken');
31
32 const obtenerRecetas = useCallback(async () => {
33   setCargando(true);
34   try {
35     const res = await axios.get('http://localhost:5000/api/recetas', {
36       headers: { Authorization: `Bearer ${token}` }
37     });
38     setRecetas(res.data);
39   } catch (err) {
40     console.error("Error al obtener recetas", err);
41     setError('No se pudieron cargar las recetas.');
```

Ilustración 17. Implementación de la lógica de consumo de la API REST mediante Axios, integrando el sistema de autenticación JWT y gestión de estados de carga.

6.5. Diseño Visual y Experiencia de Usuario (UX).

En este apartado he querido volcar mi toque personal. No buscaba una aplicación que solo funcionara bien, sino que también resulte agradable y cómoda de utilizar. ¡Qué den ganas de cocinar!

- **Identidad Visual:** He elegido una paleta de colores donde el coral es el protagonista. Un tono cálido asociado a la gastronomía que transmite energía. Esta identidad se refuerza con una jerarquía tipográfica clara mediante el uso de fuentes seleccionadas de Google Fonts, que garantizan la legibilidad en todas las secciones. He combinado este diseño con etiquetas de colores para la dificultad (verde, naranja y rojo) y una librería de iconos minimalistas de Lucide, permitiendo que, de un solo vistazo, el usuario identifique las acciones y sepa en qué ló se va a meter antes de empezar a cocinar (Lucide, s.f.); (Google Fonts, s.f.).
- **Modo Lectura Inmersivo:** Pensando en la utilidad real, he diseñado una vista específica para cuando estamos con las manos en la masa. Al activarla la letra crece y nos centramos solo en los ingredientes y los pasos. Me parece vital que no tengas que estar forzando la vista mientras cocinas.



Ilustración 18. Vista del modo lectura inmersivo.

- Feedback Visual y Efecto Confeti: Quería que la App fuera interactiva. Conforme vas terminando pasos, puedes ir tachándolos para no perderte. Y como toque final, al completar la receta, la web te felicita con un pequeño autoscroll y una lluvia de confeti. Es un detalle pequeño, pero hace que la experiencia sea mucho más divertida y gratificante (Kiril Vatev, s.f.).

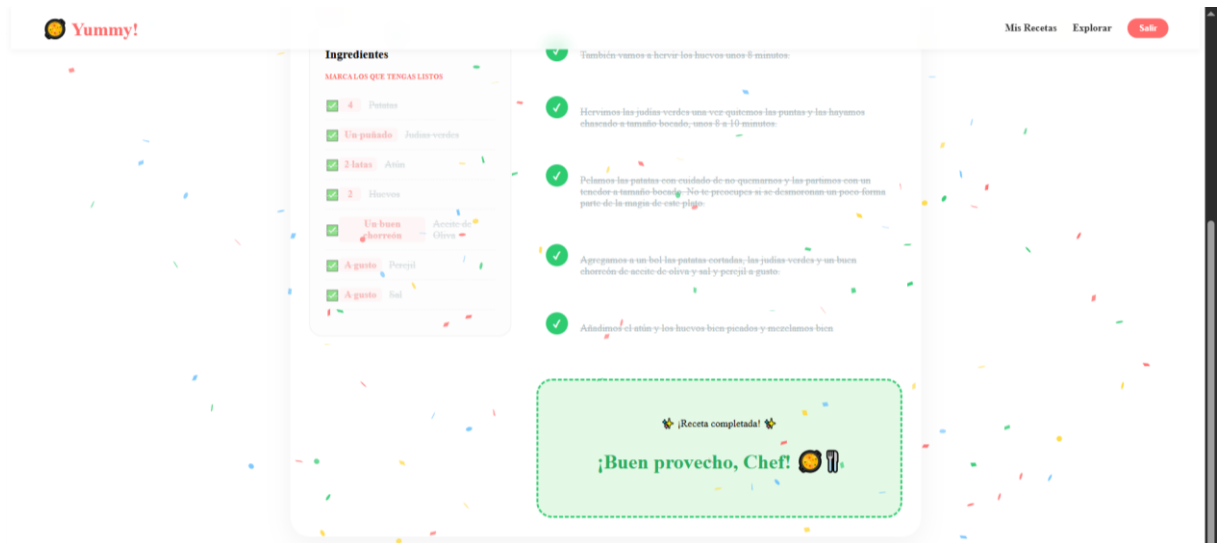


Ilustración 19. Feedback visual de éxito tras completar una receta.

Esta adaptabilidad se ha conseguido utilizando CSS Modules y Media Queries. Gracias a esto, la interfaz detecta el tamaño de la pantalla y ajusta los elementos automáticamente. Además, el Modo Lectura utiliza una lógica de estados en React que cambia los estilos para dar prioridad al contraste y al tamaño de la letra mientras se cocina.

6.6. Validación de Accesibilidad.

Para demostrar que mi compromiso con la accesibilidad no se queda solo en palabras, he sometido a Yummy! a una auditoría técnica con Lighthouse (Google Developers, s.f.), la herramienta oficial de Google. Los resultados obtenidos muestran una puntuación alta en accesibilidad, buenas prácticas y estructura general, confirmando que la aplicación cumple con los criterios que me había marcado desde el inicio (W3C, s.f.) (WCAG, s.f.).

- Puntuación de 92 en Accesibilidad: He conseguido una nota sobresaliente gracias al uso de etiquetas semánticas correctas, contrastes de color que facilitan la lectura y el uso de atributos ARIA para que cualquier lector de pantalla entienda la App.
- 100 en "Best Practices": Este pleno demuestra que el código está limpio, es seguro y sigue los estándares actuales del desarrollo web profesional.
- 100 en SEO: Significa que la aplicación está perfectamente estructurada, algo que también ayuda a la accesibilidad, ya que un sitio bien organizado para Google es un sitio bien organizado para un usuario.
- Diseño Responsive Real: Como se ve en la captura, he realizado el test simulando un dispositivo móvil. Esto confirma que Yummy! se adapta perfectamente a pantallas pequeñas, manteniendo todos los botones y menús en su sitio y listos para ser usados con una sola mano mientras se cocina.

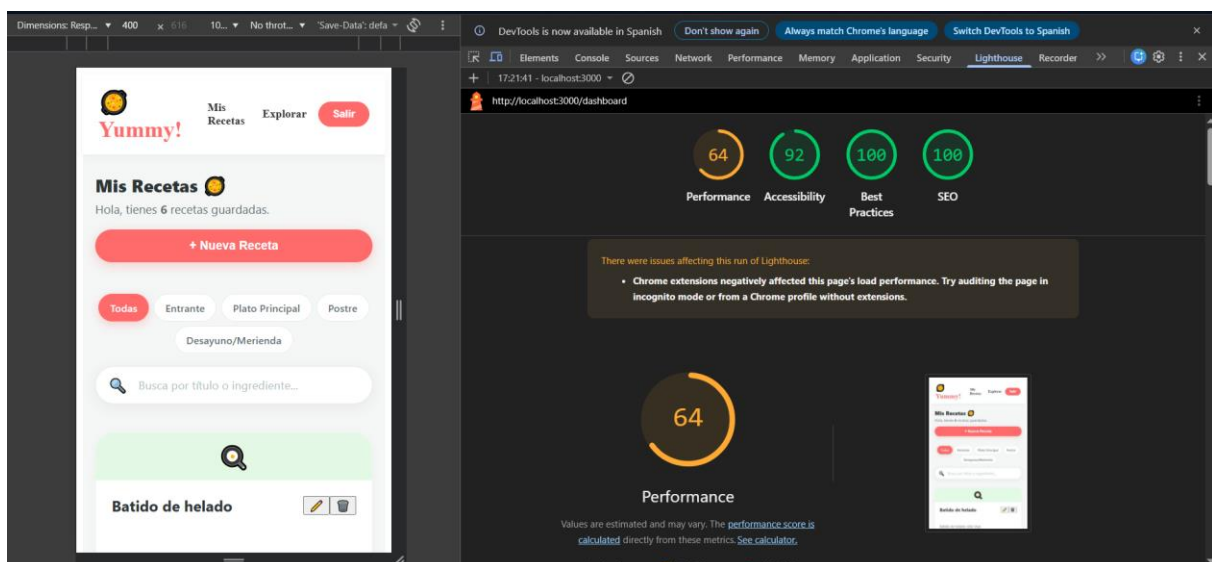


Ilustración 20. Auditoría de calidad Lighthouse: Excelencia en accesibilidad (92/100) y estándares de desarrollo (100/100).

7. Manual de Usuario.

En este capítulo se detalla el funcionamiento de la aplicación desde el punto de vista del usuario final. El objetivo de Yummy! es que cualquier persona, independientemente de su destreza tecnológica, pueda gestionar su recetario de forma sencilla.

7.1. Acceso y Registro

El acceso a la aplicación se realiza mediante un formulario sencillo donde el usuario puede registrarse o iniciar sesión. Una vez autenticado, el sistema le redirige automáticamente a su panel principal.

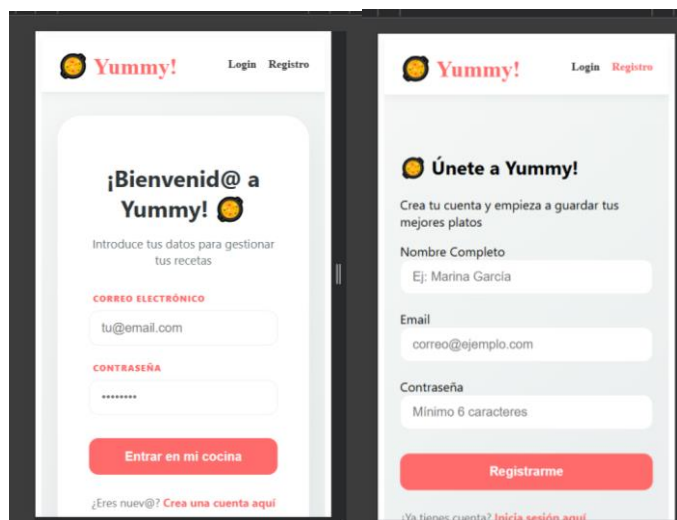


Ilustración 21. Interfaz de acceso y registro de nuevos usuarios.

7.2. Gestión de Recetas (Dashboard).

Desde el Dashboard, el usuario puede visualizar todas sus recetas y utilizar la barra de búsqueda para localizarlas rápidamente.

También puede filtrarlas por categoría y acceder directamente a las opciones de edición o eliminación.

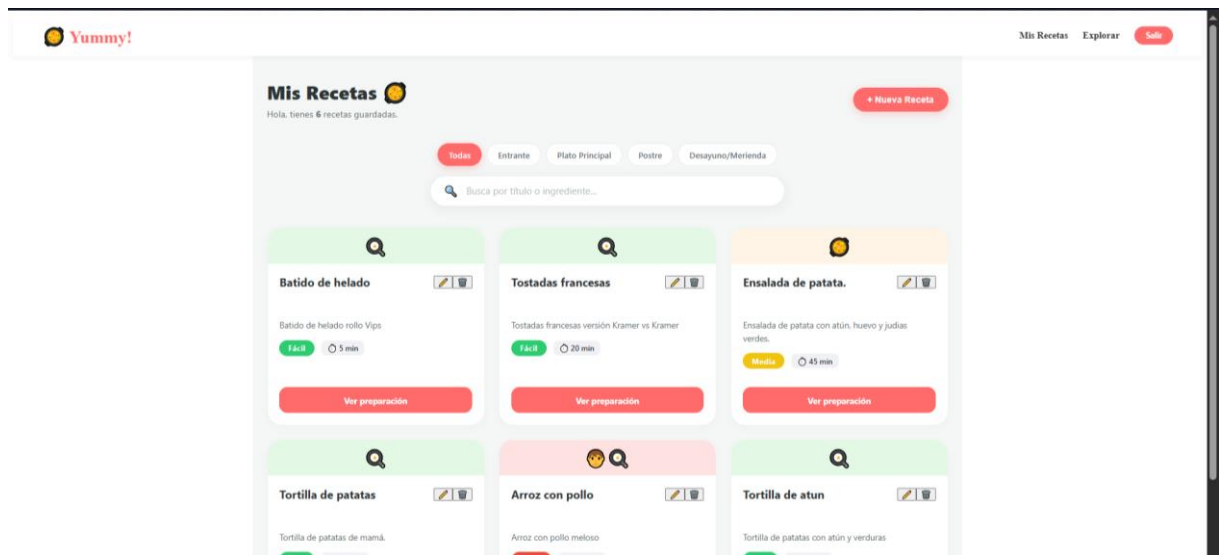


Ilustración 22. Panel principal con funciones de filtrado y búsqueda en tiempo real.

7.3. Creación y Edición de platos.

Para añadir una receta, he creado un formulario que se abre en una ventana modal (sin salir de la página). Aquí el usuario puede rellenar:

- Título y descripción.
- Listado dinámico de ingredientes y pasos (se pueden añadir tantos como hagan falta).
- Tiempo, dificultad y categoría.
- Privacidad: Un interruptor permite decidir si la receta se queda guardada solo para nosotros o si queremos que aparezca en la sección "Explorar" para toda la comunidad.

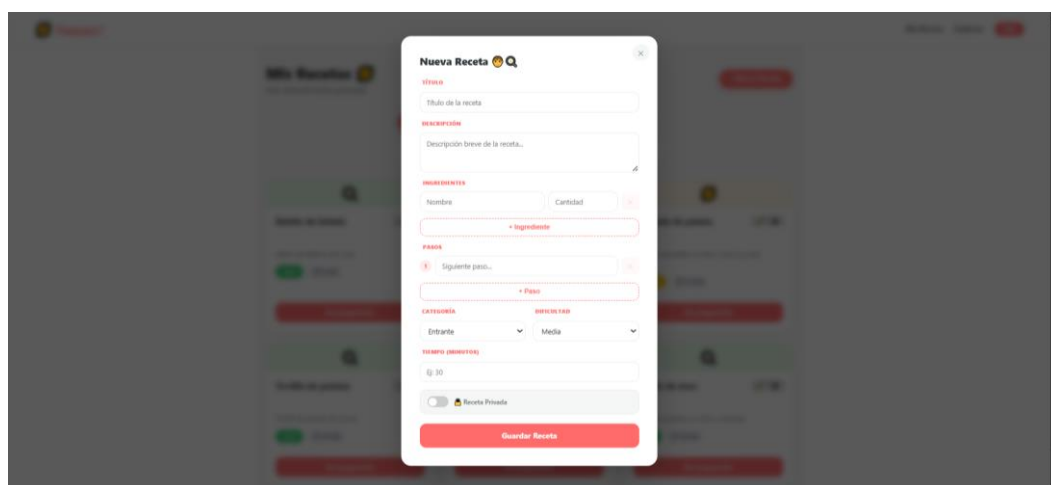


Ilustración 23. Formulario dinámico de creación de recetas con control de privacidad.

7.4. Cocinando con el Modo Lectura.

Esta es, sin duda, la funcionalidad que hace que Yummy! sea diferente. He diseñado esta vista pensando en el momento real de estar en la cocina, donde solemos tener las manos ocupadas o el dispositivo a cierta distancia.

- Claridad, ante todo: Al pulsar el botón de "Modo Lectura", se deja paso a una tipografía mucho más grande y legible.
- Interacción paso a paso: No es solo una lista estática. El usuario puede ir marcando cada ingrediente que prepara y cada paso que termina. Esto ayuda a no perder el hilo de la receta si recibimos una llamada o nos distraemos un segundo.
- Motivación visual: Al completar el último paso, la aplicación lanza una animación de confeti y un mensaje personalizado de "¡Buen provecho, chef!". Es un pequeño detalle de gamificación que hace que cerrar una receta sea un momento especial.



Ilustración 24. Interfaz del modo lectura en dispositivo móvil con feedback de finalización de receta.

7.5. Sección Explorar: Recetas de la Comunidad.

Yummy! no es solo un recetario personal, sino un espacio para compartir. En la sección Explorar, el usuario puede acceder a los platos que otros cocineros han decidido hacer públicos.

- Inspiración: En esta vista aparecen las tarjetas de recetas de otros usuarios, permitiendo descubrir nuevas ideas sin necesidad de buscarlas externamente.
- Control de Privacidad: Como he detallado en la parte técnica, el usuario decide en todo momento si su receta es "Privada" (solo la ve él) o "Pública" (aparece aquí).

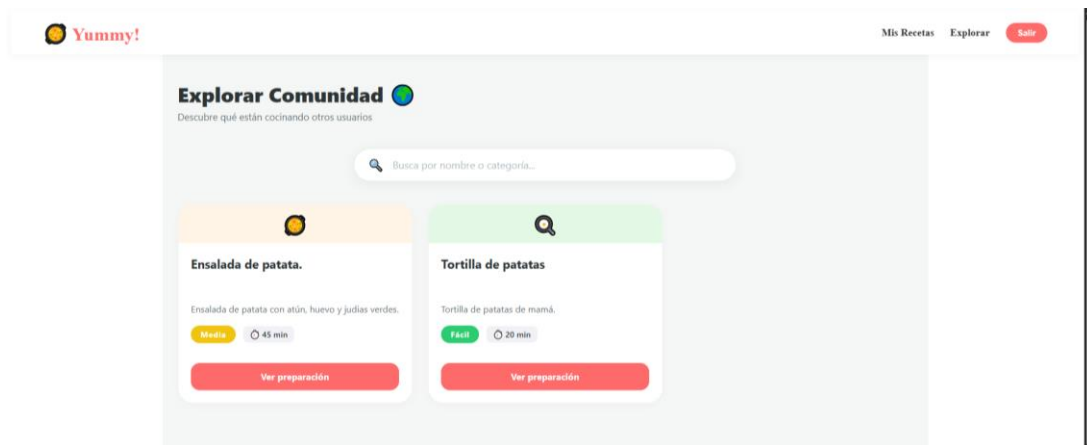


Ilustración 25. Sección de recetas públicas compartidas por la comunidad de usuarios.

8. Valoración Económica y Temporal.

8.1. Estimación temporal.

La planificación del proyecto se ha distribuido en varias fases, desde el análisis inicial hasta la validación final. Las principales etapas han sido:

- Análisis y definición de requisitos.
- Diseño de base de datos y diagramas.
- Desarrollo del backend.
- Implementación del frontend.
- Pruebas y validación.
- Documentación.

La distribución de horas refleja el peso técnico del proyecto, concentrándose principalmente en el desarrollo backend y frontend, que han supuesto más del 60% del tiempo total.

Fase del Proyecto	Descripción de tareas	Horas estimadas
Planificación y Análisis	Investigación de requisitos, creación de mockups y diagramas UML	25h
Configuración y Backend	Setup de Node.js, diseño de esquemas en MongoDB y lógica de autenticación JWT	65h
Desarrollo Frontend	Creación de componentes en React, integración con la API y lógica del Modo Lectura	60h

Pruebas y QA	Testeo de rutas en Postman y auditorías de accesibilidad con Lighthouse	15h
Documentación Final	Redacción de la memoria, manual de usuario y preparación del repositorio	15h
Total		180h

8.2. Coste estimado.

Aunque se trata de un proyecto académico, se ha realizado una estimación aproximada del coste que tendría su desarrollo en un entorno profesional.

Tomando como referencia las horas dedicadas al análisis, programación, pruebas y documentación, el coste dependería principalmente del perfil del desarrollador y del tiempo invertido.

En este caso, el uso de tecnologías open source como React, Node.js, Express y MongoDB reduce considerablemente el coste de licencias, ya que no requieren inversión adicional.

Concepto	Detalle	Coste Estimado
Mano de obra	180 horas de desarrollo full-stack a 20€ la hora	3.600€
Equipamiento	Amortización de ordenador portátil y periféricos	150€
Infraestructura Cloud	MongoDB Atlas, Hosting y Dominio (nivel inicial)	0€
Licencias Software	VS Code, Postman	0€
TOTAL		3.750€

Nota: Al ser un MVP (Producto Mínimo Viable), he aprovechado al máximo las capas gratuitas de servicios en la nube, lo que permite que el coste de mantenimiento técnico sea nulo en la fase de lanzamiento.

9. Conclusiones y Futuras Líneas de Trabajo.

9.1. Valoración del proyecto.

Yummy! ha supuesto mucho más que el desarrollo de una aplicación CRUD.

A nivel técnico, me ha permitido integrar conocimientos de backend, frontend, bases de datos, autenticación y accesibilidad dentro de un único proyecto real y funcional.

Más allá de la parte técnica, también ha sido una forma de unir mi interés por el desarrollo con algo tan cercano como la cocina y la conservación de recetas familiares.

Uno de los aprendizajes más importantes ha sido entender que una aplicación no solo debe funcionar, sino que también debe resultar cómoda, clara y agradable para quien la utiliza.

La accesibilidad, el modo lectura y el cuidado visual han sido decisiones intencionadas para que la experiencia de usuario tuviera el mismo peso que la lógica interna.

En conjunto, considero que el proyecto cumple con los objetivos planteados y representa una síntesis real de las competencias adquiridas durante el ciclo de Desarrollo de Aplicaciones Multiplataforma.

9.2. Mejoras futuras.

Aunque el proyecto es completamente funcional, existen varias líneas de mejora que podrían desarrollarse en futuras versiones.

- Incorporar imágenes propias para cada receta.
- Añadir favoritos o recetas destacadas.
- Compartir recetas entre usuarios.
- Temporizador integrado durante la preparación.
- Generación automática de lista de la compra.

Estas mejoras permitirían ampliar la utilidad de la aplicación y enriquecer todavía más la experiencia de usuario.

10. Bibliografía y Webgrafía.

Para el desarrollo del proyecto se han utilizado principalmente fuentes oficiales de las tecnologías empleadas:

Axios Project. (s.f.). Obtenido de Guía de uso para peticiones HTTP asíncronas: <https://axios-http.com/docs/intro>

Bcryptjs. (s.f.). Obtenido de Optimized bcrypt in JavaScript con zero dependencies: <https://www.npmjs.com/package/bcryptjs>

Express.js. (s.f.). Obtenido de Router: Creación de manejadores de rutas modulares y montables: <https://expressjs.com/es/guide/routing.html>

Git. (s.f.). Obtenido de <https://git-scm.com/docs/git/es>

GitHub. (s.f.). *Documentación oficial para la gestión de repositorios y control de versiones*. Obtenido de <https://docs.github.com/es>

Google Developers. (s.f.). Obtenido de Lighthouse: Herramienta de auditoría de calidad web y accesibilidad: <https://developer.chrome.com/docs/lighthouse/>

Google Fonts. (s.f.). Obtenido de Catálogo de tipografías web para diseño de interfaces legibles.: <https://fonts.google.com/>

Kiril Vatev. (s.f.). Obtenido de Documentación de la librería canvas-confetti para efectos visuales en navegador: <https://www.npmjs.com/package/canvas-confetti>

Lucide. (s.f.). Obtenido de Repositorio de iconos vectoriales optimizados para React: <https://lucide.dev/>

Meta Open Source. (s.f.). Obtenido de Documentación oficial de React: Creación de interfaces de usuario basadas en componentes: <https://react.dev/>

Microsoft. (s.f.). Obtenido de Visual Studio Code: Documentación oficial del editor de código fuente: <https://code.visualstudio.com/docs>

MongoDB. (s.f.). Obtenido de Manual de instalación y configuración de MongoDB Community Server: <https://www.mongodb.com/docs/manual/>

Mongoose. (s.f.). Obtenido de Documentación oficial de modelos y esquemas para Node.js: <https://mongoosejs.com/>

Node.js Foundation. (s.f.). Obtenido de Documentación técnica del entorno de ejecución de JavaScript: <https://nodejs.org/es/>

Postman. (s.f.). Obtenido de Manual de pruebas para servicios web y APIs: <https://learning.postman.com/>

React Router. (s.f.). Obtenido de Documentación oficial para la gestión de rutas en aplicaciones SPA: <https://reactrouter.com/>

React.js. (s.f.). Obtenido de Documentación oficial sobre Hooks (useState, useEffect, useCallback): <https://react.dev/>

Source, M. O. (s.f.). Obtenido de Documentación oficial de React: <https://react.dev/>

W3C. (s.f.). Obtenido de Pautas de Accesibilidad para el Contenido Web (WCAG) 2.1: <https://www.w3.org/WAI/standards-guidelines/wcag/es/>

WCAG. (s.f.). *Web Accessibility Initiative*. Obtenido de <https://www.w3.org/WAI/standards-guidelines/wcag/es>